ОТЧЁТ Лабораторной работе №1

---

1. Цель работы

Изучить механизмы итераторов, генераторов и сопрограмм в Python на примере генерации чисел ряда Фибоначчи.

2. Задание

1. Реализовать итератор чисел Фибоначчи двумя способами:
   - "Упрощённый" (используя только `__getitem__`)
   - "Обычный" (через методы `__iter__` и `__next__`)
2. Реализовать сопрограмму, возвращающую числа ряда Фибоначчи
3. Покрыть код тестами с использованием unittest
4. Соблюдать стандарты PEP-257 (докстринги) и PEP-484 (аннотации типов)

3. Реализация

3.1. Класс-итератор `fib_gen` (обычная версия)

Реализован через протокол итератора с методами `__iter__` и `__next__`. Использует два переменных для хранения текущих значений последовательности.

```
class fib_gen:
    """Класс-итератор чисел Фибоначи (обычная версия)"""
    def __init__(self):
        self.a = 0
        self.b = 1


    # Нужно, чтобы объект (класс) стал итератором, чтобы дальше мы смогли
обращаться к нему как к итератору
    def __iter__(self):
        return self


    def __next__(self):
        a = self.a
        self.b += self.a
        self.a = self.b - self.a
```

```
        # 0 1 1 2 3 5
        """
        a b
        0 1
        1 1
        1 2
        2 3
        3 5
        """
        return a
```

3.2. Класс `fib_gen_simplified` (упрощённая версия)

Реализован через метод `__getitem__`. Хранит последовательность в списке и динамически дополняет её при обращении по индексу.

```
class fib_gen_simplified:
    """Класс для доступа к числам Фибоначчи по индексу"""

    def __init__(self):
        self.sequence = [0, 1]  # Начальные значения

    def __getitem__(self, index):
        if index < 0:
            raise IndexError("Negative indices not supported")

        # Генерируем последовательность до нужного индекса
        while len(self.sequence) <= index:
            next_val = self.sequence[-1] + self.sequence[-2]
            self.sequence.append(next_val)

        return self.sequence[index]

    def __iter__(self):
        # Для поддержки итерации - используем генератор
        i = 0
        while True:
            yield self[i]
            i += 1
```

3.3. Генератор `fibonacchi_gen`

Функция-генератор, использующая `yield` для возврата очередного числа Фибоначчи.

```python
def fibonacchi_gen():
    """Генератор чисел Фибоначи"""
    a = 0
    b = 1

    while True:
        yield a
        res = a + b
        a = b
        b = res
```

3.4. Сопрограмма `fibonacchi_korutina`

Корутина, принимающая индекс через `send()` и возвращающая соответствующее число Фибоначчи.

```python
def fibonacchi_korutina():
    """Сопрограмма (корутина) чисел Фибоначи"""
    index = yield
    a = 0
    b = 1
    for _ in range(index):
        a, b = b, a + b
    yield a
```

3.5 Результат работы программы

```python
# pep8 и pep257 и pep484- документация
"""
подсказки при вызове функции
:param instance
"""




if __name__ == '__main__':
    n = 10
    print("Числа фибоначи через итератор\n\nОбычный вариант:")
    x = fib_gen()
    for _ in range(n):
        print(next(x))

    print("\nУпрощённый вариант:")
    y = fib_gen_simplified()
```

```
    for i in range(n):
        print(y[i])

    print("\n\nЧисла фибоначи через генератор и сопрограмму:")
    print("\nГенератор:")
    g = fibonacchi_gen()
    for i in range(n):
        print(next(g))

    print("\nЧерез корутину:")

    for i in range(n):
        h = fibonacchi_korutina()
        next(h)
        print(h.send(i))
```

```
C:\Users\Leonid\AppData\Local\Programs\Python\Python313\python.exe C:\Leo\projects\proga-4\ЛР1\fib.py
Числа фибоначи через итератор

Обычный вариант:
0
1
1
2
3
5
8
13
21
34

Упрощённый вариант:
0
1
1
2
3
5
8
13
21
34
```

```
Числа фибоначи через генератор и сопрограмму:

Генератор:
0
1
1
2
3
5
8
13
21
34

Через корутину:
0
1
1
2
3
5
8
13
21
34

Process finished with exit code 0
```

---

4. Тестирование

Разработан модуль тестов `test_fib.py` с использованием `unittest`, включающий:

- Тесты для каждой из четырёх реализаций
- Тесты граничных случаев (индексы 0, 1)
- Тест согласованности всех реализаций между собой

```
✓ 6 tests passed  6 tests total, 13 ms
C:\Users\Leonid\AppData\Local\Programs\Python\Python313\python.exe "C:/Program Files/JetBrains/PyCharm Community Edition 2025.1.1.1/plugins
Testing started at 12:47 ...
Launching unittests with arguments python -m unittest C:\Leo\projects\proga-4\ЛР1\test_fib.py in C:\Leo\projects\proga-4\ЛР1



Ran 6 tests in 0.016s

OK

Process finished with exit code 0
```

Результаты тестирования: все тесты пройдены успешно ✓

5. Выводы

1. Изучены различные способы реализации итераторов в Python
2. Получены практические навыки работы с генераторами и сопрограммами
3. Освоено написание модульных тестов с использованием unittest
4. Все реализации дают одинаковый результат и корректно работают

6. Список файлов

- `fib.py` — основная реализация
- `test_fib.py` — модуль тестов